

Recursion and tuples

LAMBDA0

CS413 · September 15, 2026

Recursive functions with T0Mfix
Structured data with pairs and projections

The current interpreter

Recursion already works in `lambda0.py`.

The pair constructors exist, but the four term operations still need cases for them.

Pair semantics in this deck describe the assignment extension.

A recursive function binds two names

f names the recursive function and x names its argument.
Both names bind occurrences in body.

```
F = TOMfix("f", "x", body)

# F is a value.
# Evaluation waits for an application.
TOMapp(F, argument)
```

Use distinct names for the function and its parameter.

Call-by-value application

Evaluate the function first, then the argument.
For $F = \text{TOMfix}(f, x, \text{body})$, substitute both values.

```
F = t0erm_cbv_evaluate0(term.arg1)
v = t0erm_cbv_evaluate0(term.arg2)

body1 = t0erm_subst0(F.arg3, F.arg2, v)
body2 = t0erm_subst0(body1, F.arg1, F)
result = t0erm_cbv_evaluate0(body2)
```

This expansion applies after checking that F is a TOMfix .

A terminating recursive example

$f(\text{flag}) = \text{if flag then } 1 + f(\text{false}) \text{ else } 0$

```
F = TOMfix("f", "flag",  
    TOMif0(TOMvar("flag"),  
        TOMop2("+", TOMint(1),  
            TOMapp(TOMvar("f"), TOMbtf(False))),  
    TOMint(0)))
```

```
t0erm_cbv_evaluate0(TOMapp(F, TOMbtf(True)))  
# TOMint(1)
```

This example uses only operators in the current interpreter.

The recursive call reaches the base case

`f(true)` evaluates the true branch.
`f(false)` evaluates the false branch.

```
f(true)
  = 1 + f(false)
  = 1 + 0
  = 1
```

`TOMif0` evaluates only the selected branch.
Unfolding a recursive function does not guarantee termination.

Binding and substitution

For $F = \text{TOMfix}(f, x, \text{body})$:

```
size(F) = 1 + size(body)
FV(F)    = FV(body) - {f, x}
```

```
subst(F, y, value):
    if y is f or x: leave F unchanged
    otherwise: substitute inside body
```

The substitution helper assumes a closed replacement term.
Use closed programs with this evaluator.

Pairs and tuple projections

A pair is a two-component tuple. Its components may have different kinds of values.

```
p = T0Mpair(T0Mint(7), T0Mbtf(True))
```

```
T0Mpfst(p)    # first component:  T0Mint(7)  
T0Mpsnd(p)    # second component: T0Mbtf(True)
```

Expected results after adding the pair cases.
These objects represent language terms, not Python tuples.

Strict evaluation of pairs

Evaluate both components, left to right.

A projection evaluates its operand and requires a pair value.

```
pair(2 + 3, 4 * 5)    evaluates to pair(5, 20)
```

```
fst(pair(5, 20))      evaluates to 5
```

```
snd(pair(5, 20))      evaluates to 20
```

```
fst(pair(1, 1 / 0))   raises ZeroDivisionError
```

```
fst(7)                raises TypeError
```

Even the component discarded by a projection must evaluate.

Notation here abbreviates the corresponding TOM constructors.

Larger tuples use nested pairs

Represent (a, b, c) as pair(a, pair(b, c)).
The chosen nesting determines the projection paths.

```
t = T0Mpair(a, T0Mpair(b, c))
```

<code>T0Mfst(t)</code>	<code># a</code>
<code>T0Mfst(T0Mpsnd(t))</code>	<code># b</code>
<code>T0Mpsnd(T0Mpsnd(t))</code>	<code># c</code>

Example use: a search state containing a board, row, and candidate.

Pairs in all four term operations

Pairs and projections introduce no binders.

$$\begin{aligned}\text{size}(\text{pair}(a,b)) &= 1 + \text{size}(a) + \text{size}(b) \\ \text{size}(\text{fst}(t)) &= 1 + \text{size}(t)\end{aligned}$$
$$\begin{aligned}\text{FV}(\text{pair}(a,b)) &= \text{FV}(a) \text{ union } \text{FV}(b) \\ \text{FV}(\text{fst}(t)) &= \text{FV}(t)\end{aligned}$$

Substitution visits the pair components or projection operand.
Evaluation constructs values and checks projections.

The second projection follows the same traversal rules as the first.

A tuple can carry recursive state

One parameter can hold several pieces of state.
Here `state = pair(flag, total)`.

```
loop(state) =  
  if fst(state)  
  then loop(pair(false, snd(state) + 1))  
  else snd(state)
```

```
loop(pair(true, 5)) = 6
```

Exercise: translate this definition using `TOMfix` and the pair constructors.
Run it after completing the pair extension.

Exercises and checks

1. Translate the recursive state example into a closed t0erm.
2. Compute size and free variables for a nested pair term.
3. Test substitution under a lambda inside a pair.
4. Test both projections and an unselected component that fails.

Integer comparisons are not yet implemented in this starter.
Add them explicitly if your eight-queens translation needs them.